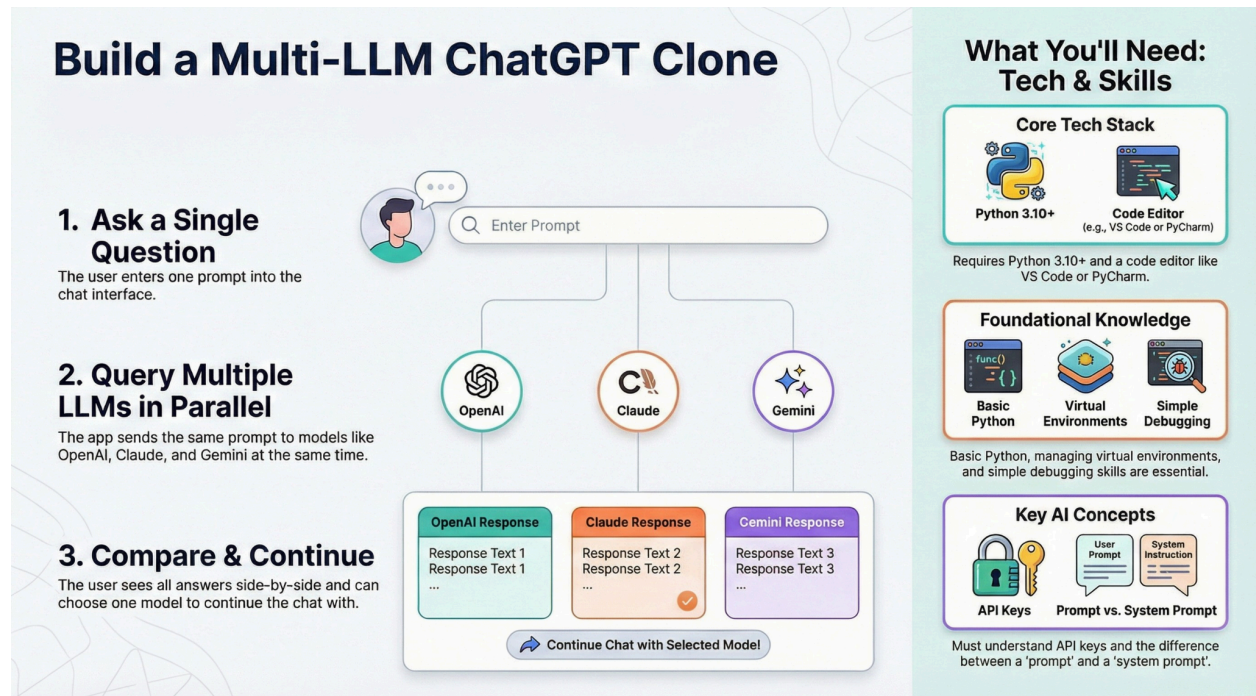


Project 1



Multi-LLM Custom ChatGPT (Side-by-Side Answer Panel)

Summary

Students will build a ChatGPT-like app where **each user question is sent to multiple LLMs in parallel** (example: OpenAI + Claude + Gemini). The UI shows **all model answers side-by-side**. The user can then click “**Continue with this model**” to keep the conversation going **with only that chosen LLM** (each model keeps its own chat history).

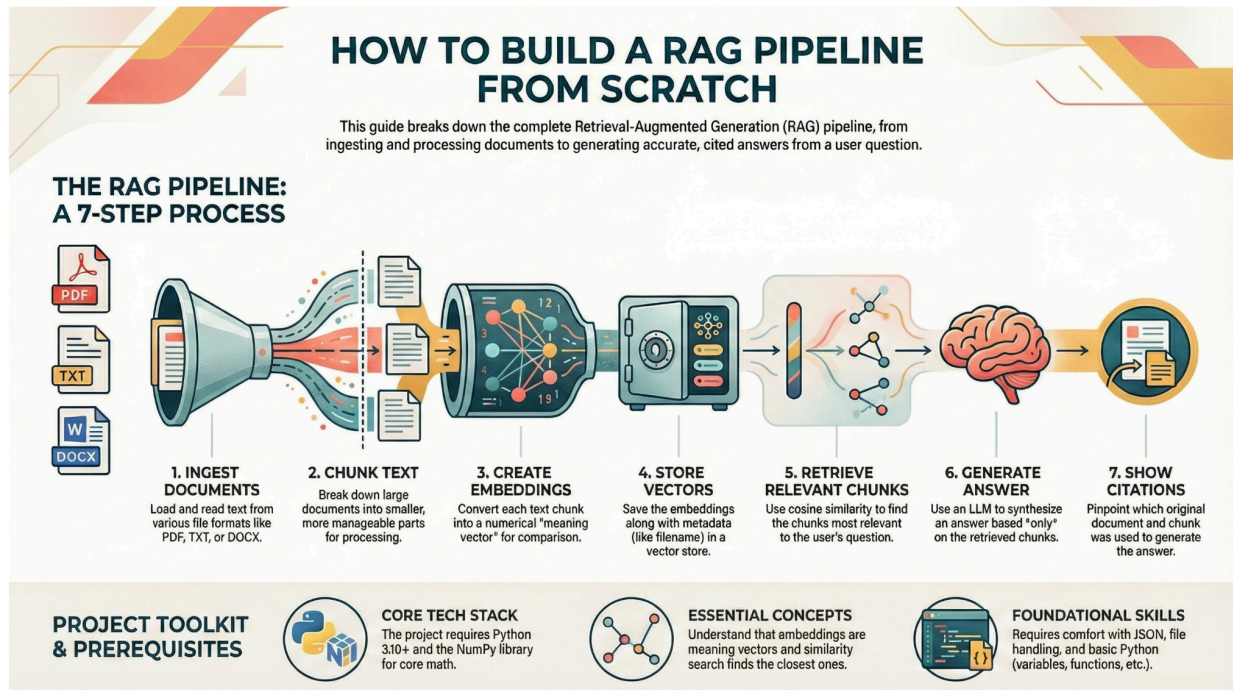
Tech Stack required

- Python 3.10+
- VS Code / PyCharm

Pre-requisites

- Basic Python: variables, functions, lists/dicts
 - Running a project: virtualenv + `pip install -r requirements.txt`
 - Understanding of:
 - “prompt” vs “system prompt”
 - API key basics (what it is, how to keep it private)
 - Basic debugging: reading error messages, file paths
-

Project 2



RAG From Scratch (Chat with Your Documents + Citations)

Summary

Students will build a document-based AI assistant using the full RAG pipeline:

1. **Ingest** documents (PDF/TXT/DOCX)
2. **Chunk** text into small parts
3. Create **embeddings** for chunks
4. Store vectors + metadata in a simple **vector store**
5. **Retrieve** top relevant chunks for a question (cosine similarity)
6. **Generate** an answer using only retrieved chunks
7. Show **citations** (which doc/chunk was used)

Tech Stack required

- **Python 3.10+**
- **VS Code / PyCharm**
- Text extraction:
 - PDF: `pypdf` or `pymupdf` (either one)
 - DOCX: `python-docx` (optional)
- Core math for retrieval: **NumPy**

Pre-requisites (for students)

- Everything from Project 1 prereqs, plus:
- Comfort with JSON and lists/dicts (metadata like filename, chunk_id)
- Basic understanding of:
 - “embedding = meaning vector”
 - “similarity search = find closest chunks”
- Handling files/folders (uploading docs, reading from disk)